МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №3

з курсу: «Мультиагентні системи»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Тема: Кооперація агентів та організація послідовної взаємодії в мультиагентних системах.

Мета роботи:

- Опанувати принципи послідовної координації (Sequential Process) в MAC.
- Реалізувати передачу контексту (Context Sharing) між агентами, де результат роботи одного агента стає вхідними даними для іншого.
- Використати патерн «Agent-Generator» для симуляції динамічного середовища та вхідних даних.

Завдання

У цій лабораторній роботі реалізовано наступний ланцюжок:

1. Генератор (Simulator): Створює "хаос" - сирі, неструктуровані дані (наприклад, плутані скарги клієнта або "брудні" логи сервера).
2. Планувальник (Architect): Аналізує "хаос" і перетворює його на "порядок". Він створює структурований об'єкт (план дій, технічне завдання JSON), який не допускає подвійного трактування.
3. Виконавець (Executor): Отримує чіткий план і виконує його по пунктах.
4. Валідатор (Reviewer): Перевіряє результат на відповідність початковим умовам.

Необхідно реалізувати MAC із послідовним типом взаємодії для вирішення складної задачі. Система повинна складатися з ланцюжка мінімум 4-х агентів, як зазначено вище і на рисунку в теоретичних відомостях до даної роботи.

Хід роботи

1. Проаналізуйте приклад 1 та його вивід, переконайтесь, що приклад коректно запускається.

Для початку ми встановили усі необхідні бібліотеки.

```
!pip install -q crewai pydantic
```

Далі виконали імпорт та налаштування (задається ключ доступу до Google Gemini API + творюється об’єкт llm, який буде використовуватися всіма агентами як мовна модель. Параметр model визначає конкретну модель Gemini, а temperature=0.5 задає помірний рівень випадковості відповідей. Це дозволяє агенту-генератору створювати не повністю однакові умови при різних запусках, але при цьому зберігати логічність і керованість результату.)

```
import os
from crewai import Agent, Task, Crew, Process, LLM
from pydantic import BaseModel, Field
from typing import List

# --- НАЛАШТУВАННЯ ---
os.environ["GOOGLE_API_KEY"] = ""

llm = LLM(
    model="gemini/gemini-3-flash-preview",
    temperature=0.5
)
```

Далі йде визначення структури даних через PydanticКлас. ProjectStep описує один окремий етап плану, який буде створювати агент-планувальник. Він містить три поля: step_name, budget_allocation та instruction. Поле step_name відповідає за назву етапу, поле budget_allocation зберігає числове значення бюджету, виділеного на цей етап, а поле instruction містить конкретну інструкцію для виконавця. Завдяки такій структурі кожен етап плану має однаковий формат, що робить результат зрозумілим і придатним для подальшої обробки іншим агентом.

Клас GrantPlan описує вже не окремий крок, а повний план реалізації проєкту. Він містить назву проєкту project_title, список етапів steps та рівень ризику risk_level. Поле steps є списком об’єктів класу ProjectStep, тобто повний план складається з декількох структурованих етапів. У цьому прикладі Pydantic-схема виконує роль формальної “мови” між агентом-планувальником і агентом-виконавцем. Планувальник повинен повернути не просто текст, а об’єкт із чітко визначеними полями, що зменшує ризик неправильного трактування результату наступним агентом.

```
class ProjectStep(BaseModel):
    step_name: str = Field(..., description="Назва етапу")
    budget_allocation: int = Field(..., description="Виділений бюджет на етап у євро")
    instruction: str = Field(..., description="Що саме треба написати або зробити")

class GrantPlan(BaseModel):
    project_title: str = Field(..., description="Згенерована назва проєкту")
    steps: List[ProjectStep] = Field(..., description="Список етапів реалізації")
    risk_level: str = Field(..., description="Оцінка ризику: Low, Medium, High")
```

Далі у нас створюються агенти.

```
grant_simulator = Agent(
    role="EU Grant Simulator",
    goal="Згенерувати складні умови конкурсу з обмеженнями.",
    backstory=(
        "Ти бюрократична машина ЄС. Ти видаєш заплутані вимоги до грантів "
        "з лімітом бюджету 50,000 EUR та вимогою екологічності."
    ),
    llm=llm,
    verbose=True
)

planner = Agent(
    role="Project Architect",
    goal="Проаналізувати умови та створити структурований план JSON.",
    backstory=(
        "Ти аналітик. Ти не пишеш воду. Ти перетворюєш вимоги на чіткі інструкції "
        "для команди, розподіляючи бюджет та задачі."
    ),
    llm=llm,
    verbose=True
)

writer = Agent(
    role="Grant Writer",
    goal="Написати текст заявки, суворо слідуючи наданому плану.",
    backstory=(
        "Ти професійний копірайтер. Ти береш JSON-план і перетворюєш його "
        "на переконливий текст. Ти не змінюєш цифри бюджету."
    ),
    llm=llm,
    verbose=True
)

auditor = Agent(
    role="Compliance Officer",
    goal="Перевірити відповідність фінальної заявки початковим умовам.",
    backstory=(
        "Ти аудитор. Якщо сума бюджету перевищує ліміт або немає згадки про екологію, "
        "ти пишеш REJECTED."
    ),
    llm=llm,
    verbose=True
)
```

Перший агент EU Grant Simulator виконує роль генератора. Його завдання полягає у створенні початкових складних і неструктурованих умов грантового конкурсу, агент імітує зовнішнє середовище, тобто створює “хаос”, який потім мають обробляти інші агенти. У його описі зазначено, що він формує заплутані вимоги до гранту з бюджетним лімітом 50 000 EUR та обов’язковою екологічною складовою.

Другий агент Project Architect виконує роль планувальника. Він отримує результат роботи генератора, аналізує його та перетворює неструктурований текст у чіткий структурований план. Його основна задача полягає в тому, щоб виділити з оголошення ключові вимоги, розподілити бюджет, сформувати етапи реалізації та підготувати інструкції для наступного агента (по суті цей агент відповідає за перетворення хаотичних даних у впорядкований Pydantic-об’єкт)

Третій агент Grant Writer є виконавцем. Він отримує вже готовий структурований план і на його основі створює повний текст проєктної заявки. Важливо, що в його описі зазначено обмеження: він не повинен змінювати цифри бюджету. Це дозволяє перевірити, чи дійсно агент-виконавець використовує переданий план, а не створює власний результат без урахування попереднього етапу.

Четвертий агент Compliance Officer виконує роль валідатора. Він перевіряє фінальну заявку на відповідність початковим умовам грантового конкурсу. Основними критеріями перевірки є дотримання бюджетного ліміту та наявність екологічної складової. Якщо заявка не відповідає вимогам, валідатор повинен видати негативний результат REJECTED. Таким чином, цей агент завершує конвеєр і оцінює коректність роботи всієї системи.

Далі йдуть таски.

```
task_simulate = Task(
    description=(
        "Згенеруй текст оголошення про грант на тему Розумні міста. "
        "Вимоги: термін 6 місяців, бюджет до 50k, участь студентів."
    ),
    expected_output="Текст оголошення.",
    agent=grant_simulator
)

task_plan = Task(
    description=(
        "Проаналізуй оголошення. Розроби план реалізації. "
        "Розбий бюджет так, щоб не перевищити ліміт."
    ),
    expected_output="Заповнений об'єкт GrantPlan.",
    agent=planner,
    context=[task_simulate],
    output_pydantic=GrantPlan
)

task_execute = Task(
    description=(
        "На основі отриманого ПЛАНУ GrantPlan напиши повний текст заявки. "
        "Опиши кожен крок згідно інструкцій."
    ),
    expected_output="Готовий текст заявки.",
    agent=writer,
    context=[task_plan]
)

task_audit = Task(
    description=(
        "Перевір фінальну заявку. Порівняй загальний бюджет з лімітом в оголошенні."
    ),
    expected_output="Звіт про відповідність.",
    agent=auditor,
    context=[task_execute, task_simulate]
)
```

Загалом цей блок демонструє механізм передачі контексту між агентами. Результат роботи одного завдання стає вхідними даними для наступного. Генератор створює початковий текст, планувальник перетворює його на структурований план, виконавець формує заявку за цим планом, а валідатор перевіряє фінальний результат (по суті така організація відповідає принципу Sequential Process, де порядок виконання завдань є критично важливим).

Наступним йде ініціалізація системи Crew.

```
crew = Crew(
    agents=[grant_simulator, planner, writer, auditor],
    tasks=[task_simulate, task_plan, task_execute, task_audit],
    process=Process.sequential,
    verbose=True
)

result = crew.kickoff()

print("### RESULT ###")
print(result)
```

Система працює як конвеєр: спочатку виконується генерація умов, потім планування, після цього написання заявки, і лише в кінці перевірка результату. Команда crew.kickoff() запускає виконання всієї системи. Після завершення роботи результат зберігається у змінній result

Результат роботи коду (повні логи див. додаток А):

```
### RESULT ###
**ЗВІТ ПРО ВІДПОВІДНІСТЬ ПРОЄКТНОЇ ЗАЯВКИ**

**Об'єкт аудиту:** Проєктна заявка «Smart Cities & Circular Synergy: Integrated Digital Ecosystem for Climate Neutrality» (Оголошення № SC-2024-DIGI-GREEN-09).
**Роль:** Compliance Officer / Аудитор.

### 1. Перевірка фінансових обмежень
*   **Максимальний ліміт гранту:** 50 000,00 EUR.
*   **Розрахунок бюджету заявки:**
    *   Крок 1 (Ініціація): 7 500 EUR
    *   Крок 2 (Закупівлі): 14 500 EUR
    *   Крок 3 (Розробка): 16 000 EUR
    *   Крок 4 (Валідація): 8 000 EUR
    *   Крок 5 (Поширення): 4 000 EUR
*   **Загальна сума:** **50 000,00 EUR**.
*   **Вердикт:** Бюджет відповідає встановленому ліміту (не перевищує 50 000,00 EUR).

### 2. Перевірка екологічних вимог
*   **Згадка про екологію:** Присутня (проєкт повністю сфокусований на екологічній стійкості).
*   **Ключові екологічні параметри в заявці:**
    *   Дотримання принципу **DNSH (Do No Significant Harm)**.
    *   Використання енергоефективного обладнання з сертифікацією **Energy Star**.
    *   Впровадження стратегії утилізації електронних відходів (**WEEE**).
    *   Цільовий показник: **скорочення вуглецевого сліду на 12%**.
    *   Відповідність стратегії «Європейський зелений курс».
*   **Вердикт:** Екологічні вимоги дотримані в повному обсязі.

### 3. Перевірка відповідності іншим технічним умовам
*   **Студентська квота:** Заявлено залучення не менше 40% студентів (відповідає вимогам конкурсу).
*   **Терміни реалізації:** Заявлено виконання протягом 182-денного дедлайну (відповідає вимогам 6 місяців).
*   **Системи звітності:** Передбачено використання eGrants та SEDIA (відповідає вимогам).
*   **Технічна документація:** Заявлено підготовку Annex I, II та III (відповідає вимогам).
```

2. Оберіть варіант завдання (наведені далі в протоколі).

Варіант. Сфера	Агент 1 (Генератор)	Агент 2 (Планувальник)	Агент 3 (Виконавець)	Агент 4 (Валідатор)
2. Cybersecurity	**Traffic Simulator** Генерує логи сервера з ознаками атаки (DDoS/SQLi).	**Incident Commander** Створює ResponsePlan (JSON): тип загрози, рівень небезпеки, список дій для блокування.	**Security Engineer** Виконує дії: пише команди для Firewall або зміни конфігурації.	**CISO** Пише звіт для керівництва про інцидент.

3. Як і в попередній роботі, використовуємо Google Colab. Необхідно встановити бібліотеки crewai та pydantic.

```
!pip install -q crewai pydantic
```

Бібліотека необхідна для імпорту класів Agent, Task, Crew, Process, LLM, BaseModel і Field для створення Pydantic-схеми.

4. Перш ніж створювати агентів, ви повинні визначити "мову", якою Агент-Планувальник спілкуватиметься з Агентом-Виконавцем. Для цього створіть клас Pydantic, який відповідає вашому варіанту завдання.

Для варіанту Cybersecurity планувальник Incident Commander має створювати ResponsePlan (структурований JSON-план реагування на інцидент), по суті цей план має містити тип загрози, рівень небезпеки та список дій для блокування

```
class ResponseAction(BaseModel):
    action_name: str = Field(..., description="Назва дії для блокування або пом'якшення атаки")
    command: str = Field(..., description="Команда Firewall або зміна конфігурації")
    reason: str = Field(..., description="Пояснення, чому ця дія потрібна")

class ResponsePlan(BaseModel):
    threat_type: str = Field(..., description="Тип загрози: DDoS або SQLi")
    severity_level: str = Field(..., description="Рівень небезпеки: Low, Medium, High або Critical")
    suspicious_ips: List[str] = Field(..., description="Список підозрілих IP-адрес із серверних логів")
    evidence_summary: str = Field(..., description="Короткий опис ознак атаки, знайдених у логах")
    actions: List[ResponseAction] = Field(..., description="Список дій для блокування або пом'якшення атаки")
```

Клас ResponseAction описує одну конкретну дію реагування на кіберінцидент. Він містить назву дії, команду Firewall або зміну конфігурації, а також пояснення, чому ця дія потрібна. Це дозволяє передавати виконавцю не загальні поради, а конкретні технічні кроки.

Клас ResponsePlan описує повний план реагування на інцидент кібербезпеки. Саме цей об’єкт створює агент Incident Commander після аналізу серверних логів. У ньому зберігається тип загрози, рівень небезпеки, список підозрілих IP-адрес, короткий опис доказів атаки та список дій для блокування або пом’якшення атаки.

Схема виконує роль мови між агентом-планувальником і агентом-виконавцем. Incident Commander перетворює сирі серверні логи на структурований ResponsePlan, а Security Engineer використовує цей план для створення команд Firewall або змін конфігурації.

5. Вам потрібно створити 4 агентів, кожен з яких відповідає за свій етап конвеєра. У backstory обов'язково вкажіть їхню роль у процесі обробки даних.

Створимо агентів

```
traffic_simulator = Agent(
    role="Traffic Simulator",
    goal="Згенерувати серверні логи з ознаками DDoS або SQLi атаки.",
    backstory=(
        "Ти агент-генератор у конвеєрі кібербезпеки. "
        "Твоя роль полягає в тому, щоб створити сирі серверні логи, "
        "які імітують реальний інцидент безпеки. "
        "Логи мають бути неструктурованими та містити ознаки DDoS або SQLi атаки."
    ),
    llm=llm,
    verbose=True
)

incident_commander = Agent(
    role="Incident Commander",
    goal="Проаналізувати серверні логи та створити структурований ResponsePlan.",
    backstory=(
        "Ти агент-планувальник у конвеєрі кібербезпеки. "
        "Твоя роль полягає в тому, щоб перетворити хаотичні серверні логи "
        "на чіткий план реагування. "
        "Ти визначаєш тип загрози, рівень небезпеки, підозрілі IP-адреси "
        "та список дій для блокування атаки."
    ),
    llm=llm,
    verbose=True
)
```

```python
security_engineer = Agent(
    role="Security Engineer",
    goal="Виконати план реагування через команди Firewall або зміни конфігурації.",
    backstory=(
        "Ти агент-виконавець у конвеєрі кібербезпеки. "
        "Ти отримуєш структурований ResponsePlan від Incident Commander "
        "і на його основі формуєш практичні команди Firewall або зміни конфігурації. "
        "Ти не повинен вигадувати нову загрозу, а маєш діяти за отриманим планом."
    ),
    llm=llm,
    verbose=True
)

ciso = Agent(
    role="CISO",
    goal="Підготувати звіт для керівництва про інцидент кібербезпеки.",
    backstory=(
        "Ти агент-валідатор у конвеєрі кібербезпеки. "
        "Твоя роль полягає в тому, щоб перевірити, чи відповідають технічні дії "
        "виявленій загрозі, та підготувати зрозумілий звіт для керівництва. "
        "У звіті потрібно описати інцидент, ризики, виконані дії та підсумковий статус."
    ),
    llm=llm,
    verbose=True
)
```

Перший агент Traffic Simulator виконує роль генератора. Він створює сирі серверні логи з ознаками атаки. Його завдання полягає в імітації динамічного середовища, де можуть виникати DDoS або SQLi інциденти.

Другий агент Incident Commander виконує роль планувальника. Він отримує логи від генератора, аналізує їх і формує структурований об’єкт ResponsePlan. Агент відповідає за перетворення хаотичних даних у впорядкований план реагування. У плані він має визначити тип загрози, рівень небезпеки, підозрілі IP-адреси та дії для блокування атаки.

Третій агент Security Engineer виконує роль виконавця. Він отримує готовий ResponsePlan і на його основі формує практичні команди Firewall або зміни конфігурації. У його описі спеціально зазначено, що він не повинен вигадувати нову загрозу, а має працювати саме з планом, який отримав від Incident Commander.

Четвертий агент CISO виконує роль валідатора. Він перевіряє, чи відповідають запропоновані технічні дії характеру інциденту, та формує звіт для керівництва.

6. Створіть завдання, пов'язавши їх у ланцюжок через параметр context. Важливо: Для завдання планування використайте параметр output_pydantic.

Створимо завдання, пов’язавши їх у ланцюжок через параметр context

```
task_generate_logs = Task(
    description=(
        "Згенеруй сирі серверні access/error логи за останні 10 хвилин. "
        "У логах мають бути ознаки однієї атаки: DDoS або SQLi. "
        "Додай IP-адреси, HTTP-методи, URL, статус-коди, timestamps та короткі фрагменти помилок. "
        "Не роби аналіз, лише згенеруй логи."
    ),
    expected_output="Сирі серверні логи з ознаками DDoS або SQLi атаки.",
    agent=traffic_simulator
)

task_create_response_plan = Task(
    description=(
        "Проаналізуй серверні логи від Traffic Simulator. "
        "Визнач тип загрози DDoS або SQLi, рівень небезпеки, підозрілі IP-адреси "
        "та створи структурований план реагування ResponsePlan. "
        "План має містити список дій для блокування або пом'якшення атаки."
    ),
    expected_output="Заповнений об'єкт ResponsePlan.",
    agent=incident_commander,
    context=[task_generate_logs],
    output_pydantic=ResponsePlan
)

task_execute_response = Task(
    description=(
        "На основі отриманого ResponsePlan підготуй конкретні технічні дії. "
        "Напиши команди Firewall або зміни конфігурації, які відповідають кожній дії з плану. "
        "Не змінюй тип загрози та список підозрілих IP-адрес із плану."
    ),
    expected_output="Список команд Firewall або змін конфігурації для реагування на інцидент.",
    agent=security_engineer,
    context=[task_create_response_plan]
)

task_ciso_report = Task(
    description=(
        "Підготуй звіт для керівництва про інцидент. "
        "Порівняй початкові логи, ResponsePlan та дії Security Engineer. "
        "Оціни, чи відповідають дії типу загрози та рівню небезпеки. "
        "Наприкінці дай статус реагування: ACCEPTED або NEEDS_REVIEW."
    ),
    expected_output="Звіт CISO для керівництва про інцидент.",
    agent=ciso,
    context=[task_generate_logs, task_create_response_plan, task_execute_response]
)
```

Перше завдання task_generate_logs передається агенту Traffic Simulator. Воно вимагає створити сирі серверні логи з ознаками DDoS або SQLi атаки. На цьому етапі агент не повинен робити аналіз, оскільки його роль полягає лише у створенні початкових даних.

Друге завдання task_create_response_plan передається агенту Incident Commander. Воно використовує результат попереднього завдання через context=[task_generate_logs]. Це означає, що планувальник отримує серверні логи від генератора і повинен на їхній основі створити структурований ResponsePlan. Тут використовується параметр output_pydantic = ResponsePlan, який змушує агента повернути результат у форматі визначеної Pydantic-схеми.

Третє завдання task_execute_response передається агенту Security Engineer. Воно отримує контекст від завдання планування через context=[task_create_response_plan]. Тобто виконавець працює не з початковими логами напряму, а з уже структурованим планом реагування. Його задача полягає у формуванні конкретних команд Firewall або змін конфігурації.

Четверте завдання task_ciso_report передається агенту CISO. Воно отримує три контексти: початкові логи, структурований ResponsePlan і дії, які сформував Security Engineer. Завдяки цьому CISO може перевірити весь ланцюжок: чи правильно визначено загрозу, чи відповідають дії плану, і чи можна вважати реагування достатнім.

7. На відміну від Лабораторної №2, тут ми використовуємо Process.sequential. Це означає, що порядок завдань у списку tasks є критичним – вони виконуватимуться зверху вниз.

```
crew = Crew(
    agents=[traffic_simulator, incident_commander, security_engineer, ciso],
    tasks=[task_generate_logs, task_create_response_plan, task_execute_response, task_ciso_report],
    process=Process.sequential,
    verbose=True
)

result = crew.kickoff()
```

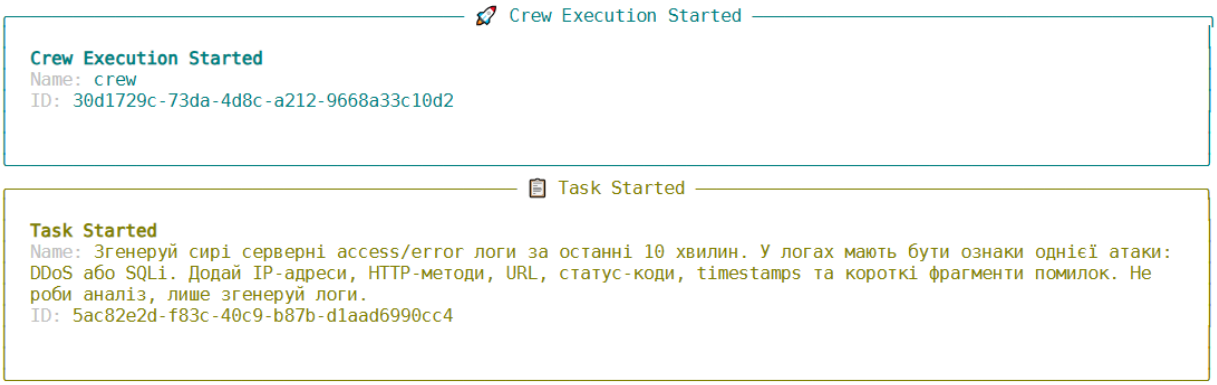
Всі агенти та завдання об’єднуються в одну систему Crew. У параметрі agents вказано чотири агенти мого варіанту: Traffic Simulator, Incident Commander, Security Engineer і CISO. У параметрі tasks вказано чотири завдання, які відповідають етапам конвеєра.

Параметр process=Process.sequential означає, що завдання будуть виконуватися строго послідовно. Спочатку генеруються серверні логи, потім вони аналізуються планувальником, після цього виконавець створює команди захисту, а наприкінці CISO формує звіт для керівництва. Такий порядок є важливим, оскільки кожен наступний агент використовує результат попереднього агента як контекст.

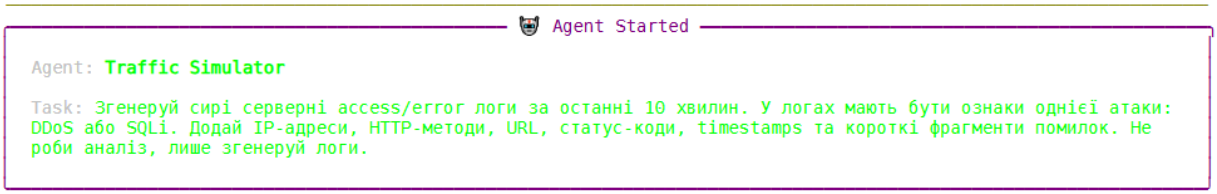
Команда crew.kickoff() запускає виконання всієї системи. Після завершення роботи фінальний результат зберігається у змінній result і виводиться на екран. Оскільки останнім етапом є звіт CISO, він і буде фінальним результатом виконання програми.

Результат роботи коду

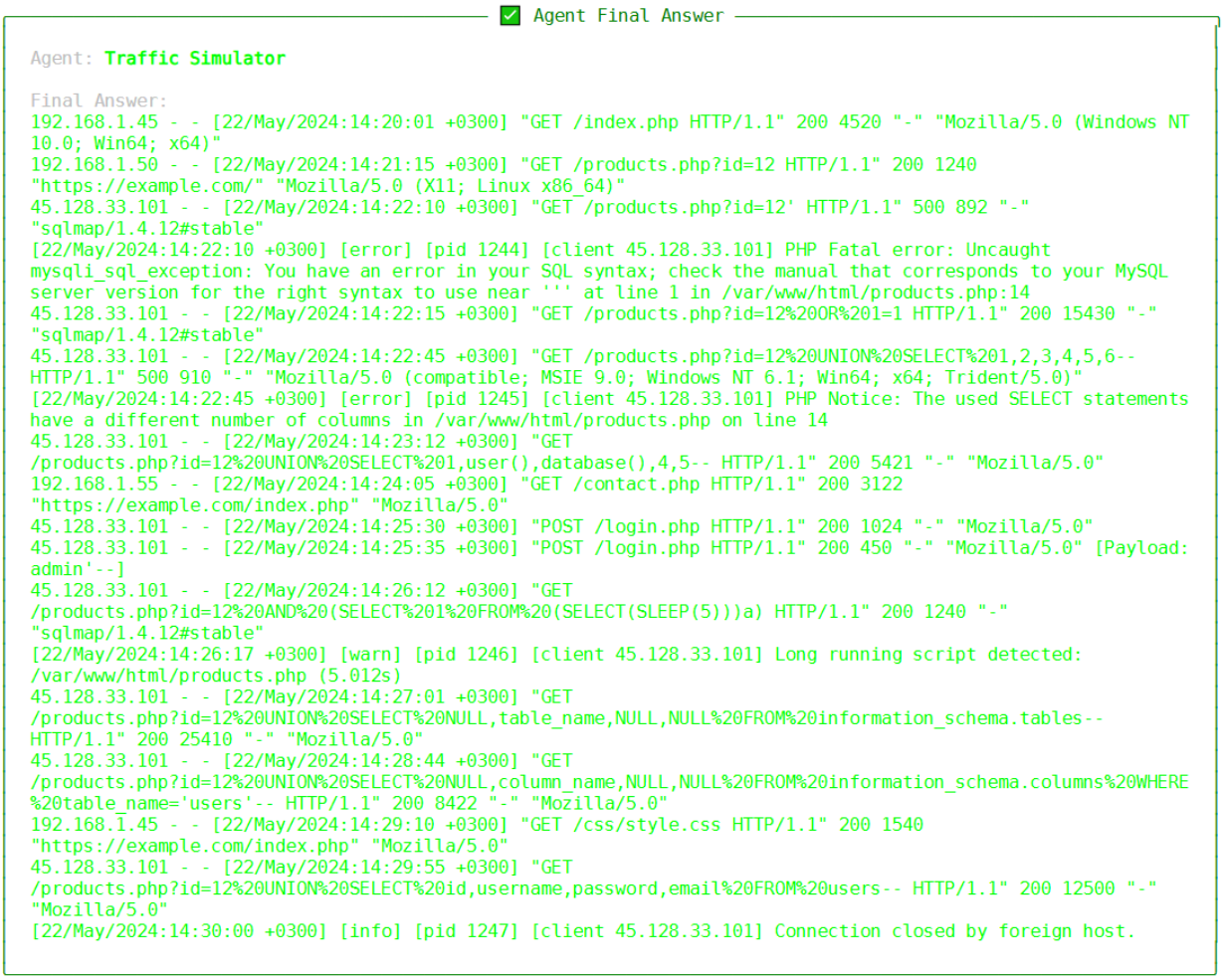
Спочатку система CrewAI розпочала виконання послідовного процесу. По логам бачимо, що з’явилося повідомлення Crew Execution Started (MC ініціалізовано)

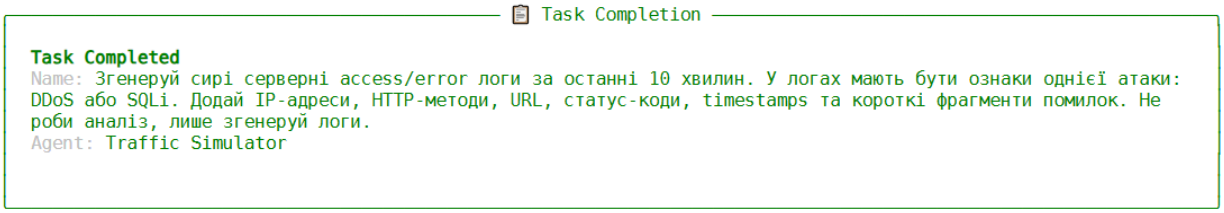


Першим запустився агент Traffic Simulator. Його завданням було згенерувати сирі серверні access/error логи за останні 10 хвилин.

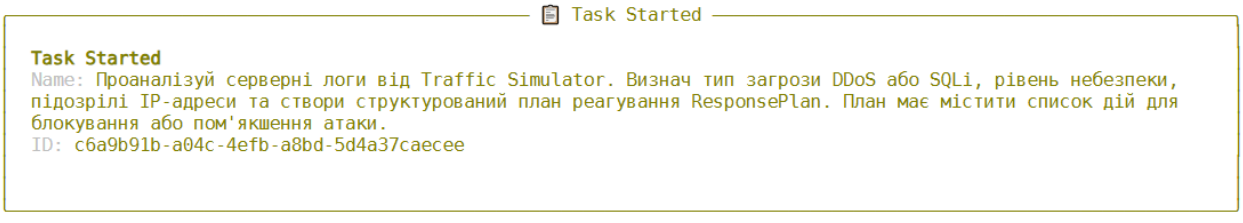


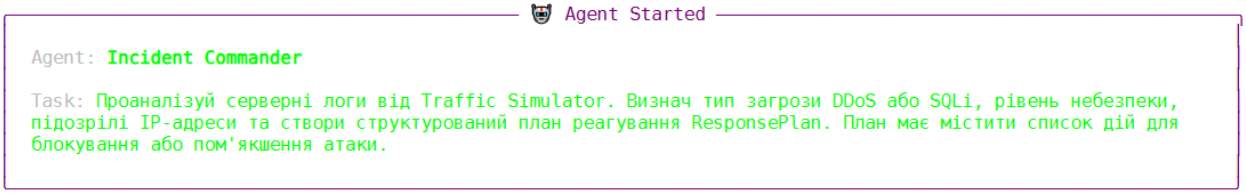
У результаті він створив набір логів, у яких є як звичайні запити від нормальних користувачів, так і підозрілі запити від IP-адреси 45.128.33.101. У логах видно запити до /products.php?id=12, використання символу ', конструкцій OR 1=1, UNION SELECT, звернення до information_schema, а також SLEEP (5). Також у User-Agent кілька разів з’являється sqlmap/1.4.12, що є явною ознакою автоматизованої SQLi-атаки.



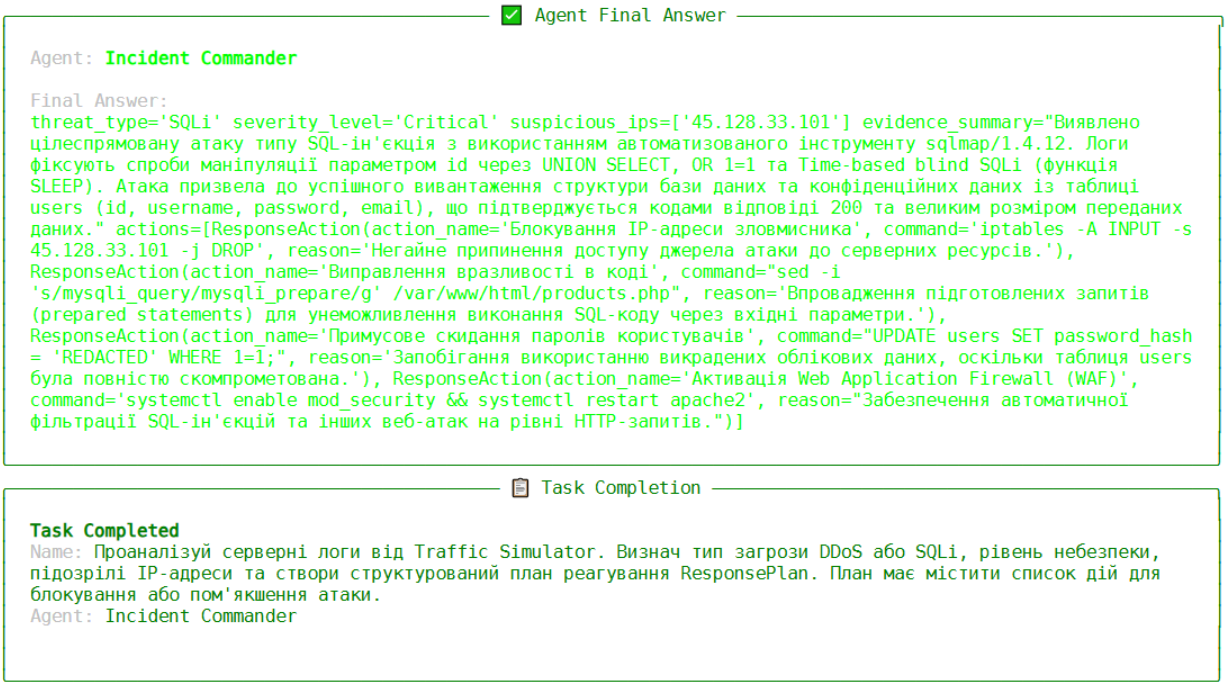


Другим запустився агент Incident Commander. Він отримав логи від Traffic Simulator через параметр context і проаналізував їх. У результаті він створив структурований об’єкт ResponsePlan.



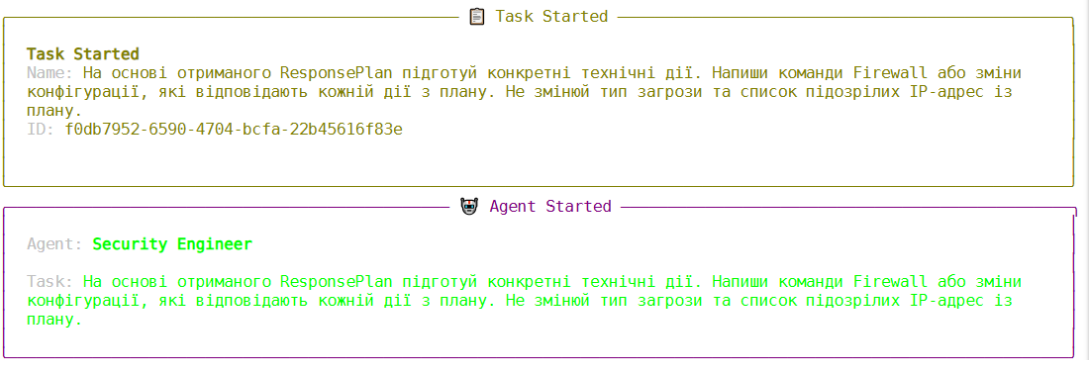


У його відповіді видно такі поля:
threat_type='SQLi'
severity_level='Critical'
suspicious_ips = ['45.128.33.101']
evidence_summary='...'
actions = [ResponseAction(...), ResponseAction(...)]

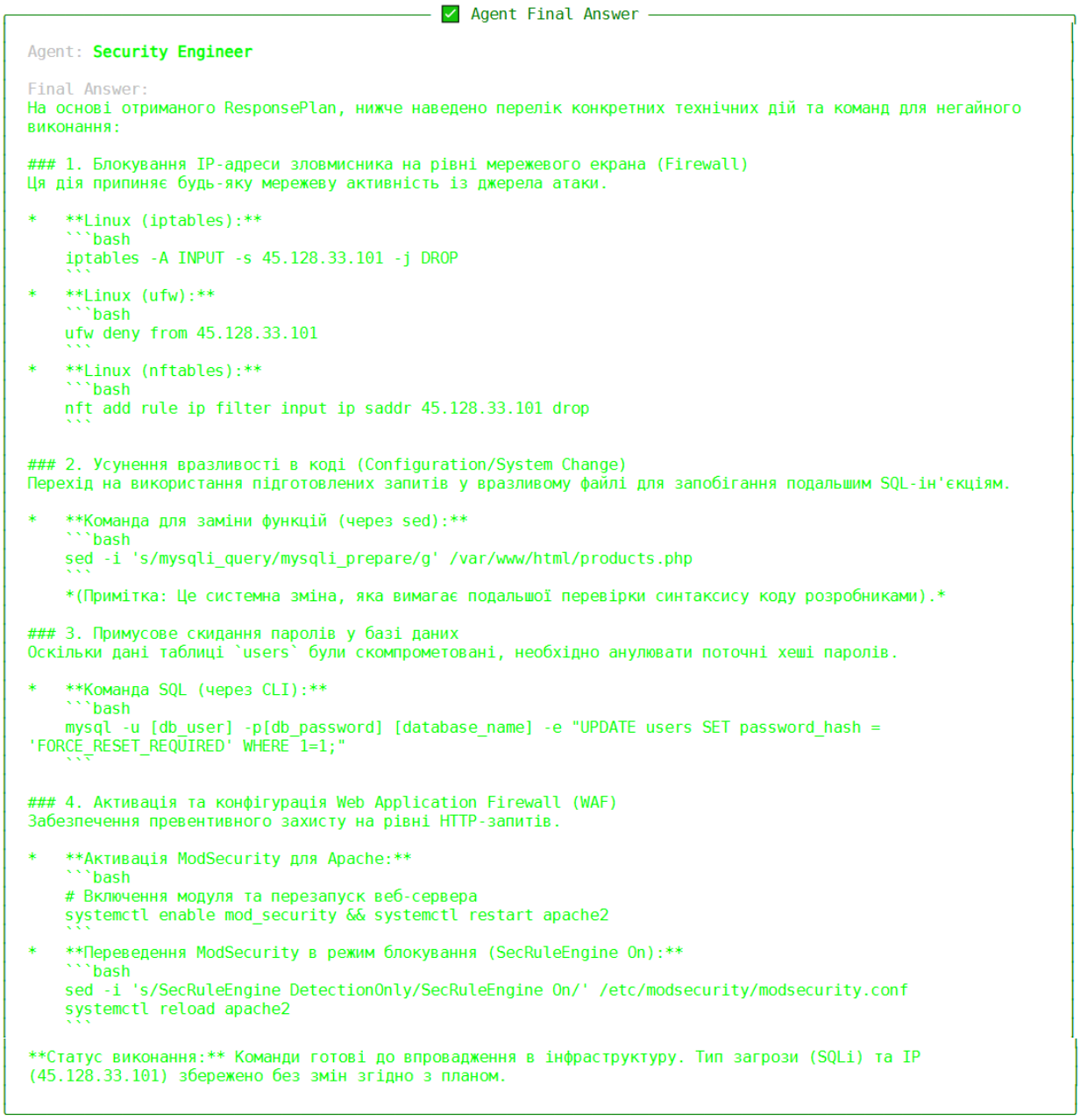


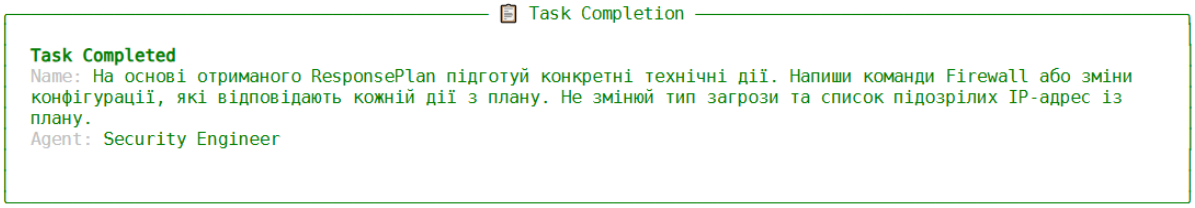
Агент Incident Commander проаналізував серверні логи та сформував структурований план реагування ResponsePlan. У цьому плані тип загрози було визначено як SQLi, рівень небезпеки як Critical, а підозрілою IP-адресою визначено 45.128.33.101. У полі evidence_summary агент пояснив, що атака містила ознаки SQL-ін’єкції: використання UNION SELECT, OR 1=1, SLEEP (5) та інструмента sqlmap. Також агент сформував список дій для реагування, зокрема блокування IP-адреси, виправлення вразливості, скидання паролів користувачів і активацію WAF.

Третім запустився агент Security Engineer. Його завданням було використати ResponsePlan і підготувати конкретні технічні дії.

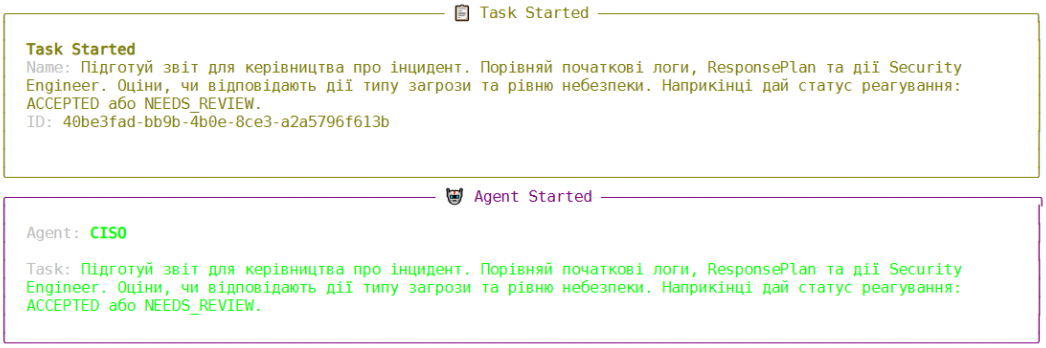


Агент Security Engineer отримав структурований ResponsePlan і на його основі сформував практичні технічні дії. Він підготував команди для блокування IP-адреси 45.128.33.101 через Firewall, запропонував виправити SQL-вразливість у файлі products.php, виконати примусове скидання паролів користувачів та активувати Web Application Firewall. Важливо, що виконавець зберіг тип загрози SQLi та підозрілу IP-адресу з плану, тобто ряльно використав інструкції агента-планувальника.

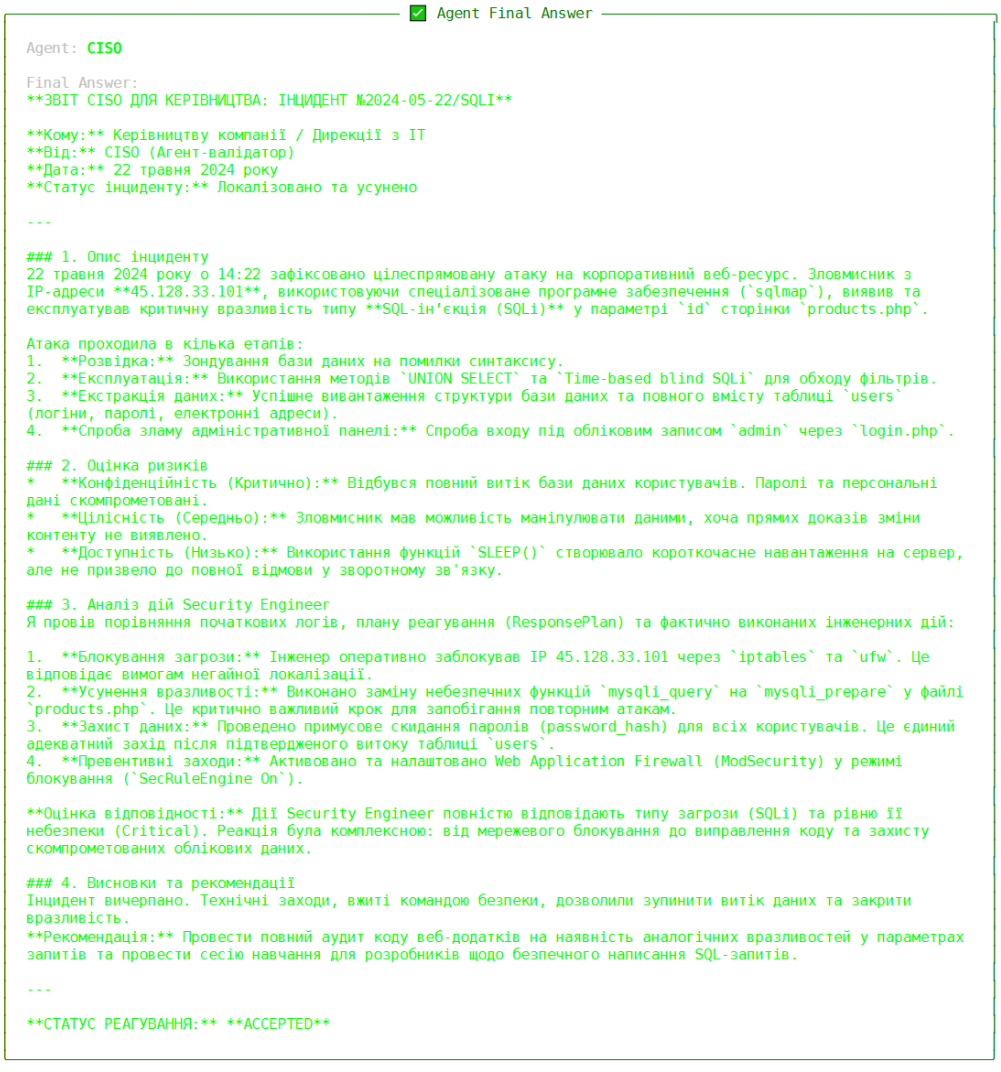


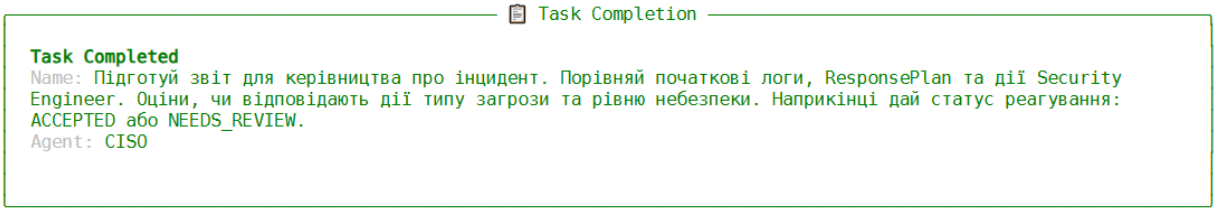


Четвертим запустився агент CISO. Він виконує роль валідатора та формує звіт для керівництва. CISO отримав одразу три контексти: початкові логи, ResponsePlan та дії Security Engineer



У звіті CISO описав інцидент як SQL-ін’єкцію, вказав IP-адресу 45.128.33.101, зазначив використання sqlmap, описав етапи атаки, оцінив ризики для конфіденційності, цілісності та доступності, а також перевірив дії Security Engineer. У кінці він зробив висновок, що дії відповідають типу загрози та рівню небезпеки, і поставив статус ACCEPTED.





Тобто, узагальнюючи, можна зазначити, що у цьому запуску мультиагентна система спрацювала правильно. Агент Traffic Simulator створив сирі серверні логи з ознаками SQLi-атаки. Агент Incident Commander проаналізував ці логи та сформував структурований ResponsePlan, у якому визначив тип загрози, рівень небезпеки, підозрілу IP-адресу та дії реагування. Агент Security Engineer використав цей план для підготовки команд Firewall і змін конфігурації. Агент CISO перевірив увесь ланцюжок і сформував звіт для керівництва зі статусом ACCEPTED.

8. Після виконання проаналізуйте логи:

a) Знайдіть момент, де Агент-Планувальник видав JSON (структуровані дані).

Агентом-планувальником є Incident Commander. Саме він отримав серверні логи від агента Traffic Simulator і сформував структурований об’єкт ResponsePlan. У логах цей момент знаходиться в блоці:

```
Agent: Incident Commander

Final Answer:
threat_type='SQLi' severity_level='Critical' suspicious_ips=['45.128.33.101'] evidence_summary="Виявлено цілеспрямовану атаку типу SQL-ін'єкція з використанням автоматизованого інструменту sqlmap/1.4.12. Логи фіксують спроби маніпуляції параметром id через UNION SELECT, OR 1=1 та Time-based blind SQLi (функція SLEEP). Атака призвела до успішного вивантаження структури бази даних та конфіденційних даних із таблиці users (id, username, password, email), що підтверджується кодами відповіді 200 та великим розміром переданих даних." actions=[ResponseAction(action_name='Блокування IP-адреси зловмисника', command='iptables -A INPUT -s 45.128.33.101 -j DROP', reason='Негайне припинення доступу джерела атаки до серверних ресурсів.'), ResponseAction(action_name='Виправлення вразливості в коді', command="sed -i 's/mysqli_query/mysqli_prepare/g' /var/www/html/products.php", reason='Впровадження підготовлених запитів (prepared statements) для унеможливлення виконання SQL-коду через вхідні параметри.'), ResponseAction(action_name='Примусове скидання паролів користувачів', command="UPDATE users SET password_hash = 'REDACTED' WHERE 1=1;", reason='Запобігання використанню викрадених облікових даних, оскільки таблиця users була повністю скомпрометована.'), ResponseAction(action_name='Активація Web Application Firewall (WAF)', command='systemctl enable mod_security && systemctl restart apache2', reason="Забезпечення автоматичної фільтрації SQL-ін'єкцій та інших веб-атак на рівні HTTP-запитів.")]
```

У виводі Colab результат показаний не у вигляді класичного JSON з фігурними дужками, а у вигляді Pydantic-об’єкта (все одно є структурованими даними) бо вони були сформовані за схемою ResponsePlan. У цьому об’єкті чітко визначено тип загрози, рівень небезпеки, підозрілу IP-адресу, короткий опис доказів атаки та список дій реагування. У моєму випадку планувальник визначив тип загрози як SQLi, рівень небезпеки як Critical, а основною підозрілою IP-адресою стала 45.128.33.101.

b) Перевірте, чи Агент-Виконавець дійсно використав інструкції з цього JSON, чи почав імпровізувати.

```
Agent: Security Engineer

Final Answer:
На основі отриманого ResponsePlan, нижче наведено перелік конкретних технічних дій та команд для негайного
виконання:

### 1. Блокування IP-адреси зловмисника на рівні мережевого екрана (Firewall)
Ця дія припиняє будь-яку мережеву активність із джерела атаки.

*   **Linux (iptables):**
    ```bash
    iptables -A INPUT -s 45.128.33.101 -j DROP
    ```
*   **Linux (ufw):**
    ```bash
    ufw deny from 45.128.33.101
    ```
*   **Linux (nftables):**
    ```bash
    nft add rule ip filter input ip saddr 45.128.33.101 drop
    ```

2. Усунення вразливості в коді (Configuration/System Change)
Перехід на використання підготовлених запитів у вразливому файлі для запобігання подальшим SQL-ін'єкціям.

*   **Команда для заміни функцій (через sed):**
    ```bash
    sed -i 's/mysqli_query/mysqli_prepare/g' /var/www/html/products.php
    ```
    (Примітка: Це системна зміна, яка вимагає подальшої перевірки синтаксису коду розробниками).

3. Примусове скидання паролів у базі даних
Оскільки дані таблиці `users` були скомпрометовані, необхідно анулювати поточні хеші паролів.

*   **Команда SQL (через CLI):**
    ```bash
    mysql -u [db_user] -p[db_password] [database_name] -e "UPDATE users SET password_hash =
'FORCE_RESET_REQUIRED' WHERE 1=1;"
    ```

4. Активація та конфігурація Web Application Firewall (WAF)
Забезпечення превентивного захисту на рівні HTTP-запитів.
```

Після формування ResponsePlan агентом Incident Commander було перевірено роботу агента Security Engineer. По логам бачимо, що він отримав структурований план реагування і сформував технічні дії саме на його основі. Виконавець зберіг тип загрози SQLi та підозрілу IP-адресу 45.128.33.101, які були визначені планувальником. Також він використав дії з плану: блокування IP-адреси через Firewall, виправлення SQL-вразливості, скидання паролів користувачів і активацію Web Application Firewall. Це підтверджує, що агент-виконавець діяв відповідно до отриманого ResponsePlan, а не створював незалежне або випадкове рішення.

c) Запустіть код ще раз (Генератор створить нові умови) і перевірте, чи адаптувався Планувальник до змін.

У другому запуску агент Traffic Simulator знову згенерував серверні логи з ознаками SQLi-атаки. Але самі умови стали іншими: змінилася дата, змінилися фрагменти логів, додалися інші SQLi-конструкції та інші ознаки атаки.

Agent Final Answer

Agent: **Traffic Simulator**

Final Answer:

```
127.0.0.1 - - [24/May/2024:14:50:02 +0300] "GET /index.php HTTP/1.1" 200 4523 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64)"
192.168.1.15 - - [24/May/2024:14:50:45 +0300] "POST /login.php HTTP/1.1" 200 1240
"http://example.com/login.php" "Mozilla/5.0 (X11; Linux x86_64)"
45.128.33.101 - - [24/May/2024:14:51:12 +0300] "GET /products.php?id=10 HTTP/1.1" 200 3120 "-" "Mozilla/5.0
(compatible; MSIE 9.0; Windows NT 6.1)"
45.128.33.101 - - [24/May/2024:14:51:15 +0300] "GET /products.php?id=10' OR 1=1 -- HTTP/1.1" 200 5421 "-"
"Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1)"
[24/May/2024:14:51:15 +0300] [error] [client 45.128.33.101] PHP Notice: Undefined variable: result in
/var/www/html/products.php on line 42
45.128.33.101 - - [24/May/2024:14:52:30 +0300] "GET /products.php?id=10' UNION SELECT 1,2,3,4,5,6 --
HTTP/1.1" 200 890 "-" "Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1)"
45.128.33.101 - - [24/May/2024:14:53:05 +0300] "GET /products.php?id=10' UNION SELECT
1,user(),database(),4,5,6 -- HTTP/1.1" 200 912 "-" "Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1)"
[24/May/2024:14:53:05 +0300] [error] [client 45.128.33.101] MySQL Error: The used SELECT statements have a
different number of columns near 'UNION SELECT 1,user(),database(),4,5,6 --'
104.24.11.5 - - [24/May/2024:14:54:20 +0300] "GET /assets/style.css HTTP/1.1" 200 15432
"http://example.com/index.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)"
45.128.33.101 - - [24/May/2024:14:55:12 +0300] "GET /products.php?id=-1' UNION SELECT 1,table_name,3,4,5,6
FROM information_schema.tables -- HTTP/1.1" 200 12450 "-" "Mozilla/5.0 (compatible; MSIE 9.0; Windows NT
6.1)"
45.128.33.101 - - [24/May/2024:14:56:45 +0300] "GET /products.php?id=-1' UNION SELECT 1,column_name,3,4,5,6
FROM information_schema.columns WHERE table_name='users' -- HTTP/1.1" 200 4500 "-" "Mozilla/5.0 (compatible;
MSIE 9.0; Windows NT 6.1)"
[24/May/2024:14:56:45 +0300] [error] [client 45.128.33.101] PHP Warning: mysqli_fetch_array() expects
parameter 1 to be mysqli_result, boolean given in /var/www/html/products.php on line 58
192.168.1.22 - - [24/May/2024:14:57:10 +0300] "GET /contact.php HTTP/1.1" 200 2100 "-" "Mozilla/5.0 (iPhone;
CPU iPhone OS 14_4 like Mac OS X)"
45.128.33.101 - - [24/May/2024:14:58:02 +0300] "GET /products.php?id=-1' UNION SELECT
1,concat(username,0x3a,password),3,4,5,6 FROM users -- HTTP/1.1" 200 6700 "-" "Mozilla/5.0 (compatible; MSIE
9.0; Windows NT 6.1)"
45.128.33.101 - - [24/May/2024:14:58:55 +0300] "GET /products.php?id=10' AND (SELECT 1 FROM
(SELECT(SLEEP(5)))a) -- HTTP/1.1" 200 3120 "-" "Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1)"
[24/May/2024:14:59:00 +0300] [warn] [client 45.128.33.101] Long running script detected:
/var/www/html/products.php, execution time: 5.01s
127.0.0.1 - - [24/May/2024:14:59:30 +0300] "GET /admin/config.php HTTP/1.1" 403 220 "-" "Mozilla/5.0 (Windows
NT 10.0; Win64; x64)"
45.128.33.101 - - [24/May/2024:15:00:01 +0300] "GET /products.php?id=10' AND 1=1 UNION SELECT
load_file('/etc/passwd'),2,3,4,5,6 -- HTTP/1.1" 500 154 "-" "Mozilla/5.0 (compatible; MSIE 9.0; Windows NT
6.1)"
[24/May/2024:15:00:01 +0300] [error] [client 45.128.33.101] SQL Error: Access denied; you need (at least one
of) the FILE privilege(s) for this operation
```

Агент Incident Commander знову визначив тип загрози як SQLi, рівень небезпеки як Critical, а підозрілою IP-адресою залишилась 45.128.33.101. Але його пояснення інциденту стало іншим, бо він врахував саме нові ознаки з другого набору логів. У evidence_summary він вказав не тільки класичні SQL-ін’єкції, а й спробу вилучення username/password, використання SLEEP і спробу читання /etc/passwd через load_file()

Agent Final Answer

Agent: **Incident Commander**

Final Answer:

```
threat_type='SQLi' severity_level='Critical' suspicious_ips=['45.128.33.101'] evidence_summary='Виявлено
цілеспрямовану атаку типу SQL Injection з IP 45.128.33.101. Логи фіксують спроби обходу авторизації (OR 1=1),
перерахування структури бази даних через UNION SELECT та information_schema, успішне вилучення даних
користувачів (username:password), використання Time-based Blind SQLi (SLEEP) та спробу читання системних
файлів (/etc/passwd) через load_file().' actions=[ResponseAction(action_name='Блокування IP-адреси
зловмисника', command='iptables -A INPUT -s 45.128.33.101 -j DROP', reason='Негайне припинення активної атаки
та запобігання подальшому витоку даних.'), ResponseAction(action_name='Впровадження підготовлених запитів
(Prepared Statements)', command='Rewrite /var/www/html/products.php using PDO or MySQLi prepared statements',
reason='Усунення першопричини вразливості шляхом безпечної обробки вхідних параметрів SQL-запитів.'),
ResponseAction(action_name='Налаштування Web Application Firewall (WAF)', command='a2enmod security2 &&
systemctl restart apache2', reason="Додатковий рівень захисту для автоматичного блокування сигнатур
SQL-ін'єкцій у майбутньому."), ResponseAction(action_name='Примусове скидання паролів користувачів',
command='UPDATE users SET password_reset_required = 1', reason='Захист облікових записів, оскільки дані
(username/password) були ймовірно скомпрометовані під час атаки.')]
```

Параметр	Перший запуск	Другий запуск
Тип атаки	SQLi	SQLi
Рівень небезпеки	Critical	Critical
Підозріла IP-адреса	45.128.33.101	45.128.33.101
Основні ознаки	sqlmap, UNION SELECT, OR 1=1, SLEEP(5), information_schema, витік users	OR 1=1, UNION SELECT, information_schema, username:password, SLEEP(5), load_file('/etc/passwd')
Дії реагування	Блокування IP, виправлення SQL-вразливості, скидання паролів, WAF	Блокування IP, Prepared Statements через PDO/MySQLi, ModSecurity WAF, прапорець скидання паролів

Висновок.

У ході виконання лабораторної роботи було реалізовано послідовну мультиагентну систему для варіанту Cybersecurity. Система складалася з чотирьох агентів: Traffic Simulator, Incident Commander, Security Engineer та CISO. Кожен агент виконував окрему роль у конвеєрі обробки інциденту. Генератор створював сирі серверні логи з ознаками атаки, планувальник аналізував ці логи та формував структурований план реагування, виконавець готував технічні команди, а валідатор перевіряв результат і створював звіт для керівництва.

Під час виконання роботи було використано Pydantic схему ResponsePlan, яка забезпечила чітку передачу даних між агентом планувальником і агентом виконавцем. Завдяки цьому Incident Commander повертав не довільний текст, а структурований план із типом загрози, рівнем небезпеки, підозрілими IP адресами, доказами атаки та діями реагування. У результатах запуску було визначено SQLi атаку з критичним рівнем небезпеки, після чого Security Engineer використав цей план для підготовки команд блокування IP адреси, активації WAF, виправлення вразливості та захисту облікових записів користувачів.

Повторний запуск програми показав, що система здатна працювати з новими вхідними умовами. Генератор створив інший набір логів, а планувальник адаптував ResponsePlan до нових ознак атаки. Це підтвердило правильну роботу механізму передачі контексту та доцільність використання Process.sequential.

Список теоретичних питань

1. Яку роль виконує бібліотека Pydantic у комунікації агентів? Чому JSON/Pydantic є кращим форматом для машинної обробки знань, ніж природна мова?

Pydantic задає чітку структуру повідомлення між агентами. У лабораторній роботі він потрібен для того, щоб агент планувальник передавав агенту виконавцю не довільний текст, а структурований об’єкт з конкретними полями. JSON або Pydantic кращі за природну мову, бо машина може однозначно прочитати поля, типи даних і значення. Природна мова може мати різні трактування, а структура типу ResponsePlan чітко показує тип загрози, рівень небезпеки, IP адреси та дії реагування.

2. Чому в даній роботі ми використовуємо Process.sequential, а не Process.hierarchical?

Бо агенти мають працювати як конвеєр. Спочатку генератор створює вхідні дані, потім планувальник їх аналізує, далі виконавець працює за планом, а в кінці валідатор перевіряє результат. Process.hierarchical більше підходить для ситуацій, де є менеджер, який керує іншими агентами. Тут головне не управління командою, а послідовна передача результату від одного агента до наступного.

3. У чому перевага використання Агента-Генератора перед завантаженням статичного файлу з даними в даній лабораторній роботі?

Перевага агента генератора в тому, що він створює нові умови при кожному запуску. Статичний файл завжди містить однакові дані, тому система просто обробляє один і той самий приклад. Агент генератор дозволяє перевірити, чи може система адаптуватися до нових логів, нових IP адрес, іншого типу атаки або інших ознак інциденту (по суті краще демонструє гнучкість мультиагентної системи)

4. Як гарантувати, що Агент-Планувальник поверне саме JSON, а не п росто текст?

Щоб агент планувальник повернув саме структурований результат, потрібно в завданні планування використати параметр output_pydantic. Наприклад, у моєму варіанті це output_pydantic=ResponsePlan. Тоді CrewAI очікує від агента відповідь, яка відповідає Pydantic схемі. Якщо модель поверне звичайний текст без потрібних полів, така відповідь не буде відповідати очікуваній структурі.

5. Якщо Агент-Виконавець працює повільніше за Агента-Планувальника, як це вплине на загальну продуктивність конвеєра? Які механізми балансування навантаження існують у MAC?

Якщо агент виконавець працює повільніше за агента планувальника, то весь конвеєр буде чекати саме його. У послідовному процесі швидкість системи обмежується найповільнішим етапом, бо наступний агент не може почати роботу без результату попереднього. У мультиагентних системах для балансування навантаження можуть використовувати паралельних виконавців, черги завдань, розподіл задач між кількома агентами, пріоритети задач і повторне призначення роботи менш завантаженим агентам.

6. Що станеться, якщо Агент-Планувальник використовує одну версію Pydantic-схеми (наприклад, поле price), а Агент-Виконавець очікує іншу (наприклад, поле cost)? Як ця проблема вирішується в класичних MAC (OWL, RDF)?

Якщо планувальник використовує одну версію схеми, наприклад поле price, а виконавець очікує інше поле, наприклад cost, виникне помилка узгодження даних. Виконавець може не знайти потрібне поле або неправильно зрозуміти план. У класичних мультиагентних системах ця проблема вирішується через спільні онтології та словники понять. OWL і RDF дозволяють формально описати, які поняття є однаковими або пов’язаними, щоб агенти однаково розуміли зміст даних.

7. Де межа між "креативністю" Агента-Планувальника (який пропонує нестандартне рішення) та "галюцинацією" (коли він вигадує неіснуючі факти)? Чи може валідатор допомогти розрізнити ці стани?

Креативність агента планувальника проявляється тоді, коли він пропонує нестандартне, але логічно обґрунтоване рішення на основі вхідних даних. Галюцинація починається тоді, коли агент вигадує факти, яких не було у вхідному контексті. Наприклад, якщо в логах немає DDoS атаки, але агент пише про DDoS, це вже помилка. Валідатор допомагає розрізнити ці стани, бо він порівнює фінальний результат із початковими даними та перевіряє, чи не з’явилися вигадані факти.

8. Агент-Генератор створює "випадкові" умови. Що таке temperature (температура генерації) і як вона впливає впливає на ступінь ентропії (хаосу) в системі?

Temperature це параметр, який впливає на випадковість відповідей мовної моделі. Низьке значення робить відповіді більш стабільними, передбачуваними та схожими між різними запусками. Високе значення збільшує різноманітність і хаотичність результатів. У цій лабораторній роботі temperature впливає на те, наскільки різними будуть умови, які створює агент генератор. Чим вища temperature, тим більша ентропія системи, тобто більше варіативності у згенерованих логах або сценаріях атаки.

Додаток А

Посилання на Google Colab:
https://colab.research.google.com/drive/1lT7etnPDIPSFXzqDBVTR8kgcHoAo5Yjc?usp=sharing